

Testbed Environment for PQC Algorithms on IoT Devices

Juan Reyes
*jereyes@mit.edu*¹

¹Massachusetts Institute of Technology / Instituto Politécnico Nacional

August 2025

Introduction

The purpose of this article is to introduce the reader to the test environment for post-quantum cryptography algorithms on IoT devices, specifically on the ESP32 microcontroller. The test environment is developed in the C programming language and uses the mbedTLS, pqm4, and liboqs libraries.

General Preparation

This article assumes that the reader has a Linux operating system. If this is not the case, refer to the [documentation](#) to install Ubuntu (Linux) [8]. Additionally, it is assumed that the reader has an environment installed for the ESP32 microcontroller. If this is not the case, refer to the [official documentation](#) from Espressif Systems [7].

Cryptographic Messages (Digital Envelopes)

Theory

When working with IoT devices, one must consider additional constraints such as energy consumption and storage space, which normally do not come into play when working with larger devices. Keeping this in mind, the implementation of asymmetric cryptography for sending messages becomes restricted due to its low speed and high computational cost.

The solution proposed to avoid excessive use of asymmetric algorithms is the implementation of hybrid cryptography, which, in this context, involves the use of both symmetric and asymmetric encryption. Under this idea, digital envelopes are developed.

Suppose Alice wants to send Bob a message. To send a message $\mathbf{m}$ using a digital envelope, Alice must first generate the key $\mathbf{k}$ for the symmetric algorithm, e.g., AES. Bob, for his part, must generate the public key $\mathbf{pk}$ and secret key $\mathbf{sk}$ of the asymmetric algorithm, e.g., RSA, and share $\mathbf{pk}$ with Alice.

Alice then encrypts $\mathbf{m}$ using $\mathbf{k}$ to obtain the ciphertext $\mathbf{c}$. It is important to note

that at this point Bob does not know k , so Alice must, in some way, include this information so that Bob can obtain m from c . However, sending k unprotected would allow any adversary to obtain m . To avoid this, Alice encrypts k using pk to obtain the encrypted key k' and sends the envelope containing $c \parallel k'$.

At this point, Bob uses his secret key sk to decrypt k' and recover k . After recovering k , Bob can now decrypt c to recover m , thus completing the exchange.

It is important to see that this protocol reduces the use of asymmetric encryption to a single key whose size is small compared to a message whose size is essentially unrestricted. This process is also known as a key encapsulation mechanism (KEM), a process that will become more important later.

On the other hand, it is also important to know that in the implementation of different algorithms additional information such as initialization vectors can be included. Likewise, the envelope may contain additional information such as the type of algorithms used, in case this was not agreed upon beforehand. The simplified version is shown in Algorithms 1 and 2 below, where it has already been agreed to use the algorithms KEM and Sim.

Algorithm 1 Create Digital Envelope

Input: Message m ; recipient's public key pk

Output: Digital envelope $\mathcal{E}$.

- 1 $(c) \leftarrow \text{Sim.Encrypt}(k, m)$
 - 2 $(k') \leftarrow \text{KEM.Encapsulate}(pk, k)$
 - 3 $\mathcal{E} \leftarrow \{c, k'\}$
 - 4 **return** $\mathcal{E}$
-

Algorithm 2 Open Digital Envelope

Input: Digital envelope $\mathcal{E}$

Output: Message m

- 5 $(c, k') \leftarrow \mathcal{E}$
 - 6 $(k) \leftarrow \text{KEM.Decapsulate}(sk, k')$
 - 7 $(m) \leftarrow \text{Sim.Decrypt}(k, c)$
 - 8 **return** m
-

Implementation

To implement the digital envelopes, the mbedTLS library was used, which is already included as a component within ESP-IDF. Additionally, the RSA public and private keys used in the project were generated using OpenSSL. To learn how to create RSA keys, refer to the following [forum](#). The full implementation can be accessed through the following GitHub [link](#) [5]. It is recommended that the reader reset the device port after obtaining the repository of our implementation.

In the implementation, the presence of a file named *partitions.csv* stands out, which provides the specifications for allocating space in the ESP32 for the keys stored in the *partition* directory. Additionally, the *setup* directory includes general functions such as storage creation, setup of the entropy counter required for mbedTLS projects, proper formatting of messages to be processed using AES, and reading a digital envelope component by component. As a final note, observe that in *main/CMakeLists.txt* there is an extra line necessary for creating the storage space where the RSA keys will be stored.

PQC Algorithms

Theory

Digital envelopes are widely used today; however, the most commonly used asymmetric encryption algorithms, such as RSA and DH, are susceptible to attacks by quantum computers using Shor’s algorithm. In this context, NIST proposed a competition for creating and standardizing algorithms that are resistant to quantum attacks. While quantum computers are not expected to be capable of breaking today’s algorithms for quite some time, it is still imperative to protect current data against “collect now, decrypt later” attacks.

This is how different key encapsulation mechanisms (KEMs) and digital signature algorithms (DSAs) presumed to be resistant to quantum attacks arise and become popular; hence they are called post-quantum cryptography (PQC) algorithms. Among these algorithms stand the KEM family Kyber and the DSA family Dilithium (both lattice-based), which come in versions satisfying NIST security levels 1, 3, and 5, corresponding to security levels present in symmetric algorithms such as AES128, AES192, and AES256 respectively. However, these new algorithms present new challenges, especially when focusing on constrained devices such as IoT devices.

The various algorithms proposed for PQC security have key, message, and signature sizes much larger than their classical counterparts. Likewise, current hardware lacks specific optimizations for these algorithms—optimizations that do exist for popular algorithms such as RSA. Therefore, depending on the device, it is possible that several of the theoretically secure proposals have tradeoffs that make them non-viable or, in extreme cases, entirely non-implementable.

Implementation

To implement the PQC algorithms, the pqm4 library and its PQCclean version were used. This library is one of the main bases of the OQS (Open Quantum Safe) project and

provides implementations of various families of PQC algorithms. The pqm4 version is designed for ARM-Cortex microcontrollers, but PQCclean includes C implementations specifically designed to allow development on other architectures. For this project, round 3 of the PQCclean repository was used, accessible through the following [link](#) [2]. The full implementation can be accessed through the following GitHub [link](#) [3]. It is recommended that the reader reset the device port after obtaining the repository of our implementation.

Note that the implementation does not contain the entire PQCclean library, only the files necessary for the current program to run. This was done to maximize memory efficiency. Most of the directories present in the implementation are direct copies from the PQCclean directories, but this is not the case for all files—for example, the *fips202.** files come from the *common* directory in PQCclean. If more algorithms are to be added to this implementation, it will be necessary to import the corresponding extra directories and files.

This implementation first aimed to run a KEM algorithm, so the simplest in the Kyber family—Kyber512—was chosen. Initially, it could not run due to issues with the size of the ESP32’s main stack, so it had to be increased to 12KB. After this adjustment, the algorithm successfully ran and the project expanded to implement various DSA families.

These algorithms presented similar difficulties and required increasing the stack size again. Additionally, the time required to run all tests in series is too high, so the ESP32’s WatchDogTimer (WDT) commonly activates. If the terminal shows a related message, do not worry—the program is still running and about to finish. Nevertheless, disabling the WDT is recommended.

In the main program, *main.c*, the commented values of the stack size necessary to run each algorithm can be found. Note that these values are not strict; smaller values may work as long as if two algorithms have different required sizes, the larger one cannot run with the smaller size. During this process it was also useful to determine the largest stack size needed, resulting in 179712 bytes. The results regarding the necessary stack size are shown in Table 1.

Algorithm Family	Algorithm	Required Stack Size
KEM Kyber	Kyber512	12288
DSA Dilithium	Dilithium2	65536
	Dilithium3	131072
	Dilithium5	179712
DSA Falcon	Falcon512	65536
	Falcon1024	131072
DSA Sphincs	Sphincs128	32768
	Sphincs192	65536
	Sphincs256	65536

Cuadro 1: Stack size required in bytes per algorithm on ESP32 using PQCclean

Comparison with Literature

The creation of the following test environment was inspired by the results of Abbasi et al. [1], who conducted a comparative performance evaluation of several PQC algorithms on various devices including IoT devices. Based on that, the same library used by Abbasi et al., liboqs, available through the following GitHub link [6], was used. The full implementation can be accessed through the following GitHub link [4]. It is recommended to reset the device port after obtaining the repository of our implementation.

Liboqs is a library that is part of the Open Quantum Safe (OQS) project, which aims to integrate and develop applications secure against quantum computing [6]. This library builds on PQClean, and thus includes various optimizations for ARM32v7, ARM64v8, and x86-64 architectures. However, it also includes a generic C version to work across architectures.

To work with this library on a different architecture, it is necessary to prepare the *cmake* files so that they accept the processor, compiler, and initialization process when building from the source code provided in the GitHub repository. This preparation can be found in our implementation under the name *toolchain-esp32-xtensa.cmake* and must be placed in the root directory of the cloned liboqs repository when building the files.

We emphasize the build process for two reasons. First, the amount of memory used can be optimized depending on how the library will be used. This can be done through various “flags.” The specific flags of our implementation can be found under the name *flags.txt*. Among these configurations, it is important to note that although the intention was to include the KEM BIKE family in the comparative evaluation, this was not possible because the library does not allow its implementation on 32-bit architectures such as the ESP32. Second, in order for the implementation to run, a file specifying how the library was built is required; in our implementation, this file is located at *components/oqs-prebuilt/lib/liboqs.a* and must be replaced if the library is rebuilt—for example, when enabling more algorithm families.

Finally, this implementation evaluates the KEM families Kyber, ML-KEM (updated version of Kyber), and NTRU, and the DSA families Dilithium and Falcon. Unlike the previous test environment, here the average time required to generate keys, encapsulate, and decapsulate (for KEMs), and to generate keys, sign, and verify (for DSAs) is compared. Additionally, the standard deviation of the obtained data is included. To obtain the results, the three evaluated steps per algorithm were run in series a total of 1000 times before continuing to the next algorithm. The results can be seen in Table 2. As with the PQClean implementation, the reader is strongly encouraged to disable the WDT because these tests require significant time.

It is interesting to observe the overwhelming difference between the results in [1] and those obtained by us. Some explanations could be:

- Abbasi et al. used an IoT device with an ARM Cortex-A53 processor, which has optimizations within liboqs, whereas the ESP32 uses an Xtensa processor, which does not have such optimizations.

- Abbasi et al. used an IoT device with 1GB of RAM, while we used one with only 530KB of RAM, which may lead to memory fragmentation issues.
- Abbasi et al. used an IoT device with a processor running at 1.4 GHz, while ours runs at 240 MHz, affecting the number of operations per millisecond, among other factors.

Nevertheless, more research is needed to determine where these differences specifically come from, as better understanding them would allow us to develop more appropriate protocols for devices with more constraints, unlike those that, while still within the IoT context, have far superior capabilities.

Family	Algorithm	Subroutine	Execution Time	Standard Deviation
Kyber	Kyber512	KeyGen	162.54	4.35
		Encaps	204.80	5.00
		Decaps	165.77	4.98
	Kyber768	KeyGen	259.73	3.20
		Encaps	330.67	2.54
		Decaps	275.03	5.12
	Kyber1024	KeyGen	416.76	5.32
		Encaps	504.95	5.12
		Decaps	432.98	4.57
ML-KEM	ML-KEM-512	KeyGen	157.75	6.39
		Encaps	164.52	6.62
		Decaps	200.14	5.46
	ML-KEM-768	KeyGen	263.54	6.73
		Encaps	264.48	7.45
		Decaps	317.34	7.30
	ML-KEM-1024	KeyGen	401.66	8.42
		Encaps	408.89	8.90
		Decaps	479.15	8.75
NTRUPrime	sntrup761	KeyGen	883.39	4.73
		Encaps	61.71	3.77
		Decaps	108.57	3.50
Dilithium	Dilithium2	KeyGen	619.76	9.07
		Sign	1221.39	655.14
		Verify	591.74	3.79
	Dilithium3	KeyGen	1125.38	4.99
		Sign	1960.91	923.95
		Verify	1040.91	2.88
	Dilithium5	KeyGen	1938.39	12.04
		Sign	2774.79	910.22
		Verify	1857.62	4.26
Falcon	Falcon512	KeyGen	15245.92	13986.01
		Sign	2774.79	3.95
		Verify	1857.62	4.26
	Falcon1024	KeyGen	26903.48	23898.73
		Sign	768.76	3.33
		Verify	122.58	4.38

Cuadro 2: Time required (ms) to execute subroutines in PQC algorithms

Acknowledgment

I am deeply grateful to Dr. Moises Salinas for guiding me throughout the development of this article, as well as the development of the entire project and my time at the National Polytechnic Institute in Mexico City.

Referencias

- [1] Maryam Abbasi, Filipe Cardoso, Paulo Váz, José Silva, and Pedro Martins. A practical performance benchmark of post-quantum cryptography across heterogeneous computing environments. *cryptography*, 2025.
- [2] PQClean. Pqclean round3. <https://github.com/PQClean/PQClean/releases/tag/round3>, 2023.
- [3] Juan Reyes. Implementacion-pqclean. <https://github.com/jereyes-Mondrano/Implementacion-PQClean>, 2025.
- [4] Juan Reyes. liboqs-testbed. <https://github.com/jereyes-Mondrano/liboqs-testbed>, 2025.
- [5] Juan Reyes. Sobres-digitales. <https://github.com/jereyes-Mondrano/Sobres-Digitales>, 2025.
- [6] Open Quantum Safe. liboqs. <https://github.com/open-quantum-safe/liboqs>, 2025.
- [7] Espressif Systems. Esp-idf programming guide. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>, 2025.
- [8] Canonical Ubuntu. Install ubuntu desktop. <https://ubuntu.com/tutorials/install-ubuntu-desktop>, 2025.